CSE4065 – Fall 2022
Computational Genomics
Genome Assemble

Slides are from:

Bioinformatics Algorithms: An Active Learning Approach

String Composition Problem

- Assume we are given a string and we want to obtain its k-mer composition
  - The collection of all k-mers forming the string (including repeated *k*-mers)
- Representation: $Composition_k(Text)$
- What is $Composition_3$(TATGGGGTGC)?
- $Composition_3$(TATGGGGTGC) = {ATG, GGG, GGG, GGT, GTG, TAT, TGC, TGG}.
- *k*-mers are listed in **lexicographic order** rather than in the order of their appearance in TATGGGGTGC.
- The correct ordering of the reads is unknown when they are generated.

The String Reconstruction Problem – Example I

- Consider the following example of a 3-mer composition:

  AAT ATG GTT TAA TGT

- Which 3-mer can be used to start the string?
- The string should start with TAA because there is no 3-mer ending in TA.

- Which 3-mer should be selected next?
- The next 3-mer in the string should start with AA. There is only one 3-mer satisfying this condition, AAT

The String Reconstruction Problem – Example I cnt.

- Consider the following example of a 3-mer composition:

  AAT ATG GTT TAA TGT

- The first and second 3-mers are:
- TAA AAT

- How to generate the string?

TAA
AA**T**
AT**G**
TG**T**
GT**T**
TAATGTT

The String Reconstruction Problem – Example II

- Consider the following example of a 3-mer composition:

AAT ATG ATG ATG CAT CCA GAT GCC GGA GGG GTT TAA TGC TGG TGT

- Construct the string: **TAATGCCATGGGATGTT**

TAA
AAT
ATG
TGC
GCC
CCA
CAT
ATG
TGG
GGG
GGA
GAT
ATG
TGT
GTT

The String Reconstruction Problem with Repeating k-mers

- How can we solve the problem if we have k-mers that repeat multiple times?

- Repeats in a genome necessitate some way of looking ahead to see the correct assembly in advance.

- We can use graphs to solve this problem.
  - Find a Hamiltonian path in the **overlap graph** - find a path visiting every *node* exactly once
  - Find an Eulerian path in the **de Bruijn graph** - find a path visiting every *edge* exactly once

Eulerian graphs

- Euler's Theorem states that the following two conditions are sufficient to guarantee that an arbitrary graph is Eulerian.

- Any Eulerian graph must be balanced:
  - The number of incoming edges at any node must be equal to the number of outgoing edges at that node.
  - We define the **indegree** and **outdegree** of a node v (denoted $IN(v)$ and $OUT(v)$) as the number of edges leading into and out of v.
  - A node v is **balanced** if $IN(v) = OUT(v)$
  - A graph is **balanced** if all its nodes are balanced.

- Any Eulerian graph must be strongly connected:
  - A directed graph is **strongly connected** if it is possible to reach any node from every other node.

Euler's Theorem

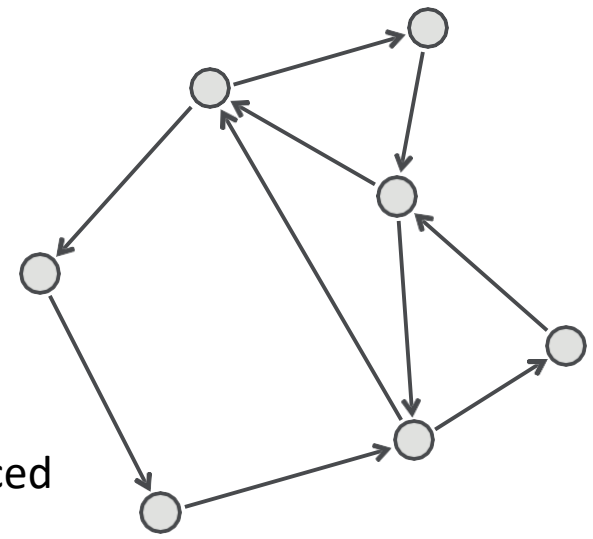
Euler's Theorem: Every balanced, strongly connected directed graph is Eulerian.

Euler's Theorem for Directed Graphs

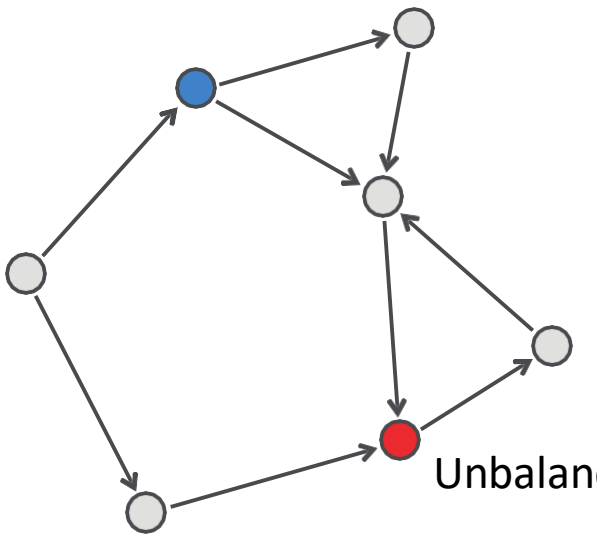
Indegree: Number of edges leading into a node.

Outdegree: Number of edges leading out of a node.

Balanced graph: Every node has indegree equal to outdegree.



Balanced

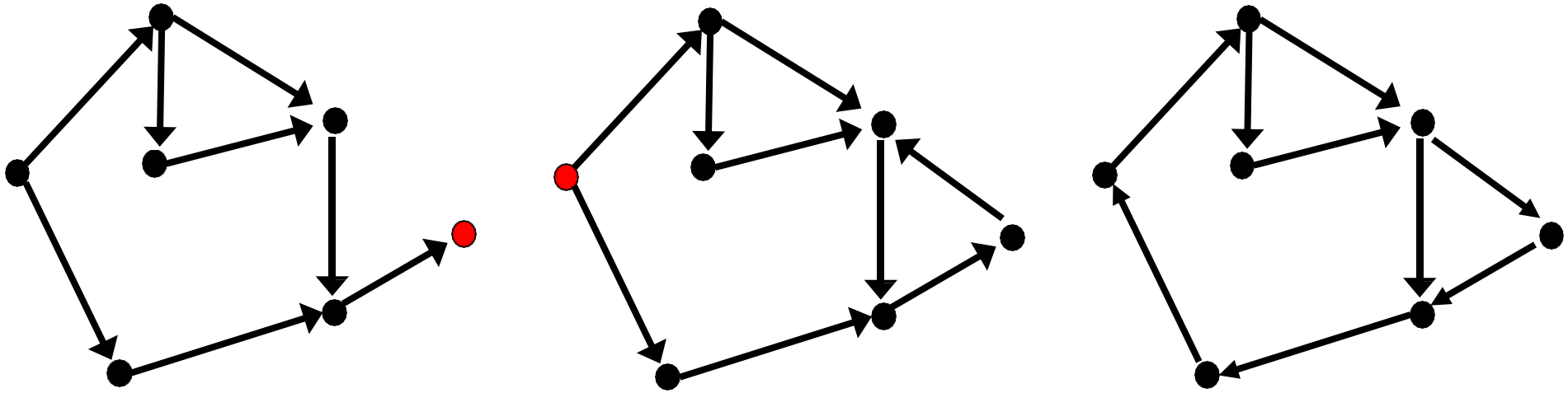


Unbalanced

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

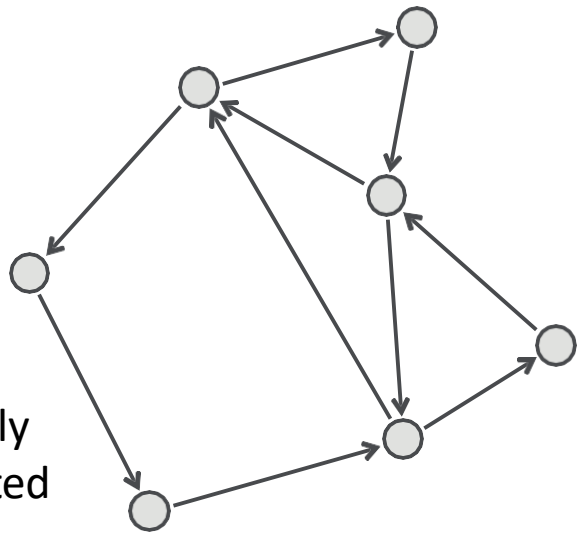
A Graph is **Eulerian** if It Contains an Eulerian Cycle.

Is this graph Eulerian?

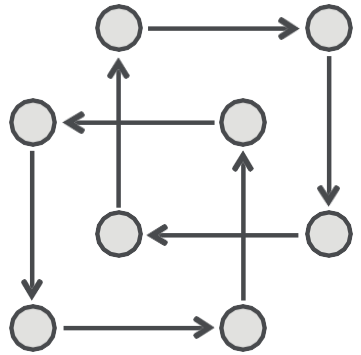


Euler's Theorem for Directed Graphs

Strongly connected graph: A graph where it is possible to reach every node from any other node.



Strongly connected



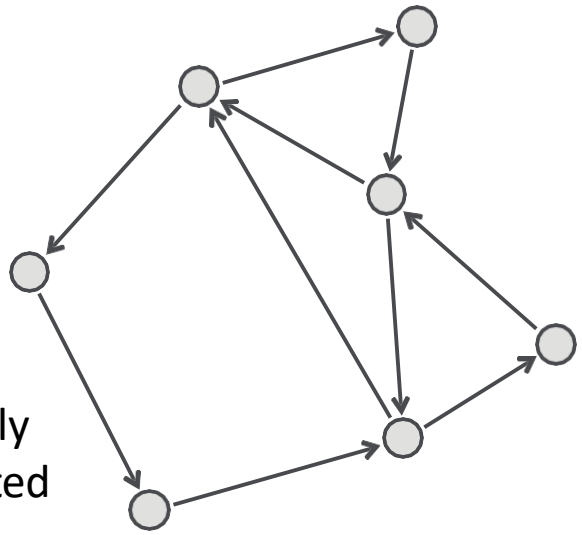
Disconnected

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

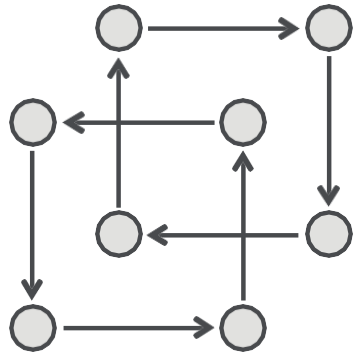
Euler's Theorem for Directed Graphs

Strongly connected graph: A graph where it is possible to reach every node from any other node.

Euler's Theorem: Every balanced, strongly connected graph has an Eulerian cycle.



Strongly connected

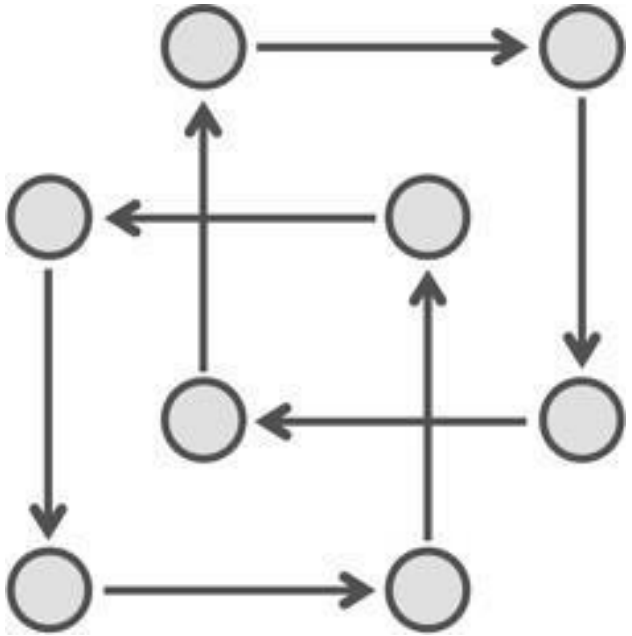


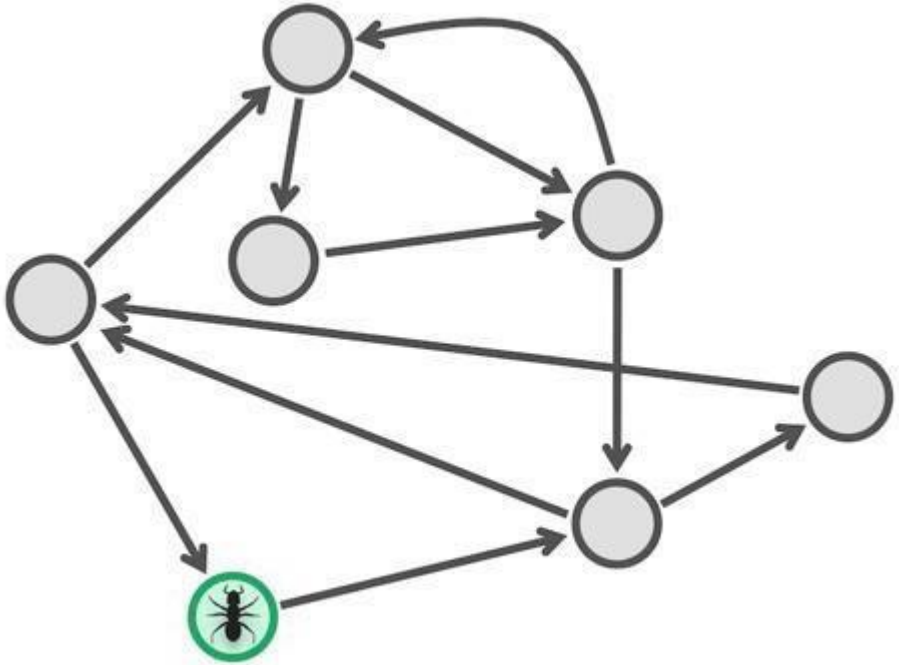
Disconnected

Strongly connected graphs

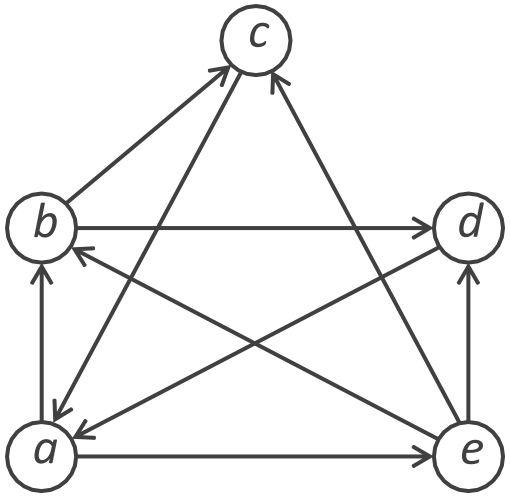
- A directed graph is **strongly connected** if it is possible to reach any node from every other node.

- Are the following graphs strongly connected and balanced?





Two Ways to Represent Graphs Computationally



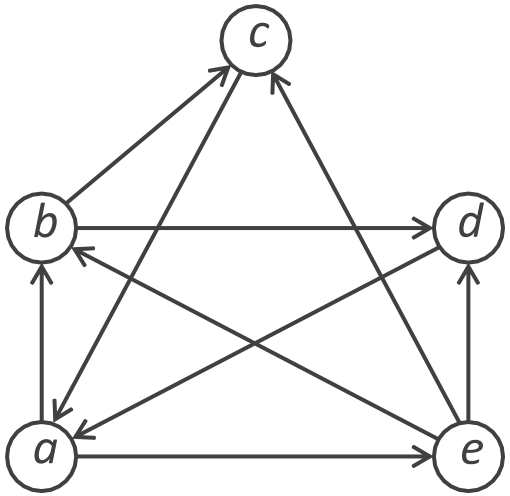
Adjacency Matrix

	a	*b*	*c*	*d*	*e*
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency matrix: $A_{i,j} = 1$ if there is an edge connecting node i to node j; $A_{i,j} = 0$ otherwise.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Two Ways to Represent Graphs Computationally



Adjacency Matrix

	a	*b*	*c*	*d*	*e*
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency List

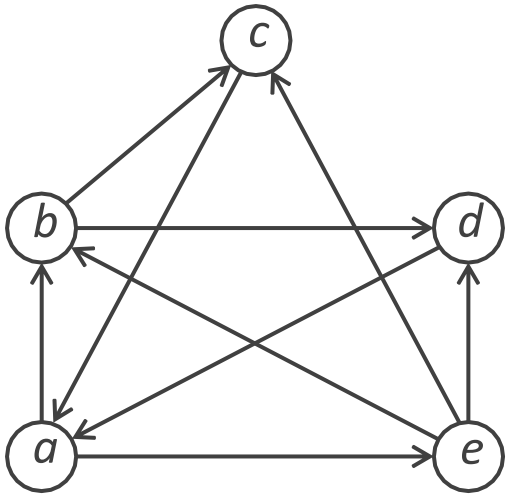
a	*b, e*
b	*c, d*
c	*a*
d	*a*
e	*b, c, d*

Adjacency matrix: $A_{i,j} = 1$ if there is an edge connecting node i to node j; $A_{i,j} = 0$ otherwise.

Adjacency list: Dictionary; "key" node i;"value" is list of nodes that i is connected to.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

How Changing a Graph Changes Adjacency Matrix/List

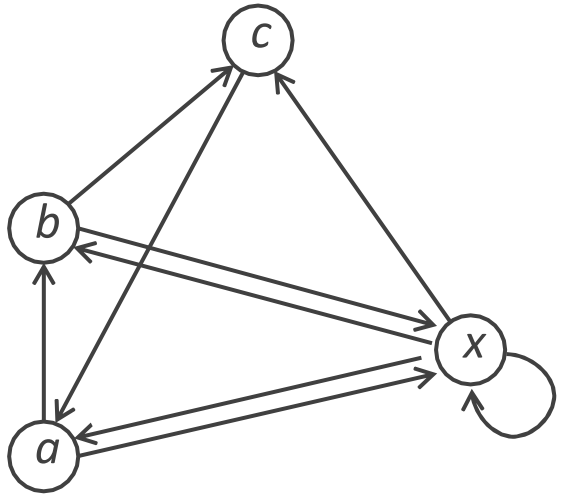


Adjacency Matrix

	a	*b*	*c*	*d*	*e*
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency List

a	*b, e*
b	*c, d*
c	*a*
d	*a*
e	*b, c, d*



Adjacency Matrix

	a	*b*	*c*	*x*
a	0	1	0	1
b	0	0	1	1
c	1	0	0	0
x	1	1	1	1

Adjacency List

a	*b, x*
b	*c, x*
c	*a*
x	*a, b, c, x*

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Similar Problems with Different Fates

Hamiltonian Cycle Problem

NP-Complete

Input: a network with *n* nodes.

Output: "Yes" if there is a cycle visiting every ***node*** in the network; "No" otherwise.

Eulerian Cycle Problem

Polynomial

Input: a network with *n* nodes.

Output: "Yes" if there is a cycle visiting every ***edge*** in the network; "No" otherwise.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Euler's Theorem is "Constructive"

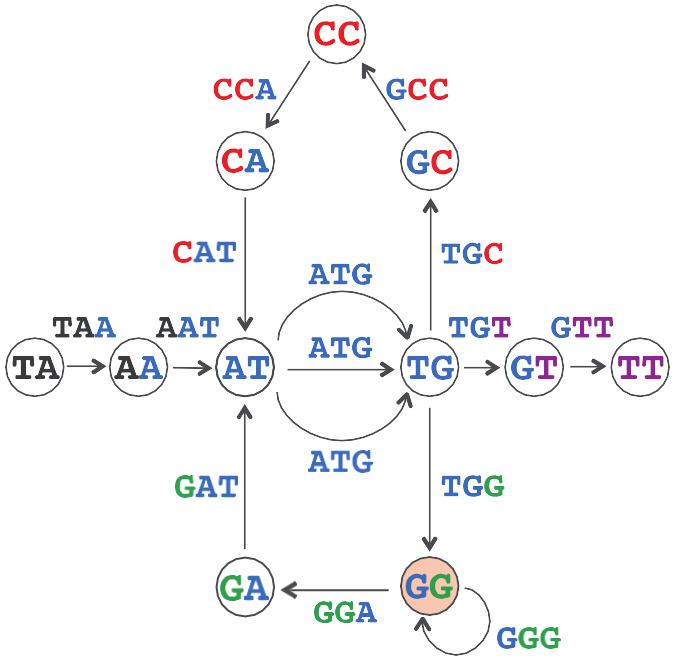
Key Point: This is a "constructive proof", meaning it implies an algorithm for finding an Eulerian cycle.

```
EulerianCycle(Graph)
    v ← arbitrary node in Graph
    Cycle ← randomly walk starting at v (don't revisit edges) until cycle
    while there are unexplored edges in Graph
        newStart ← node in Cycle with unexplored edges
        Cycle' ← cycle formed by traversing Cycle (starting at newStart)
                 and then randomly walking
        Cycle ← Cycle'
  return Cycle
```


Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

From Eulerian Cycles to Paths

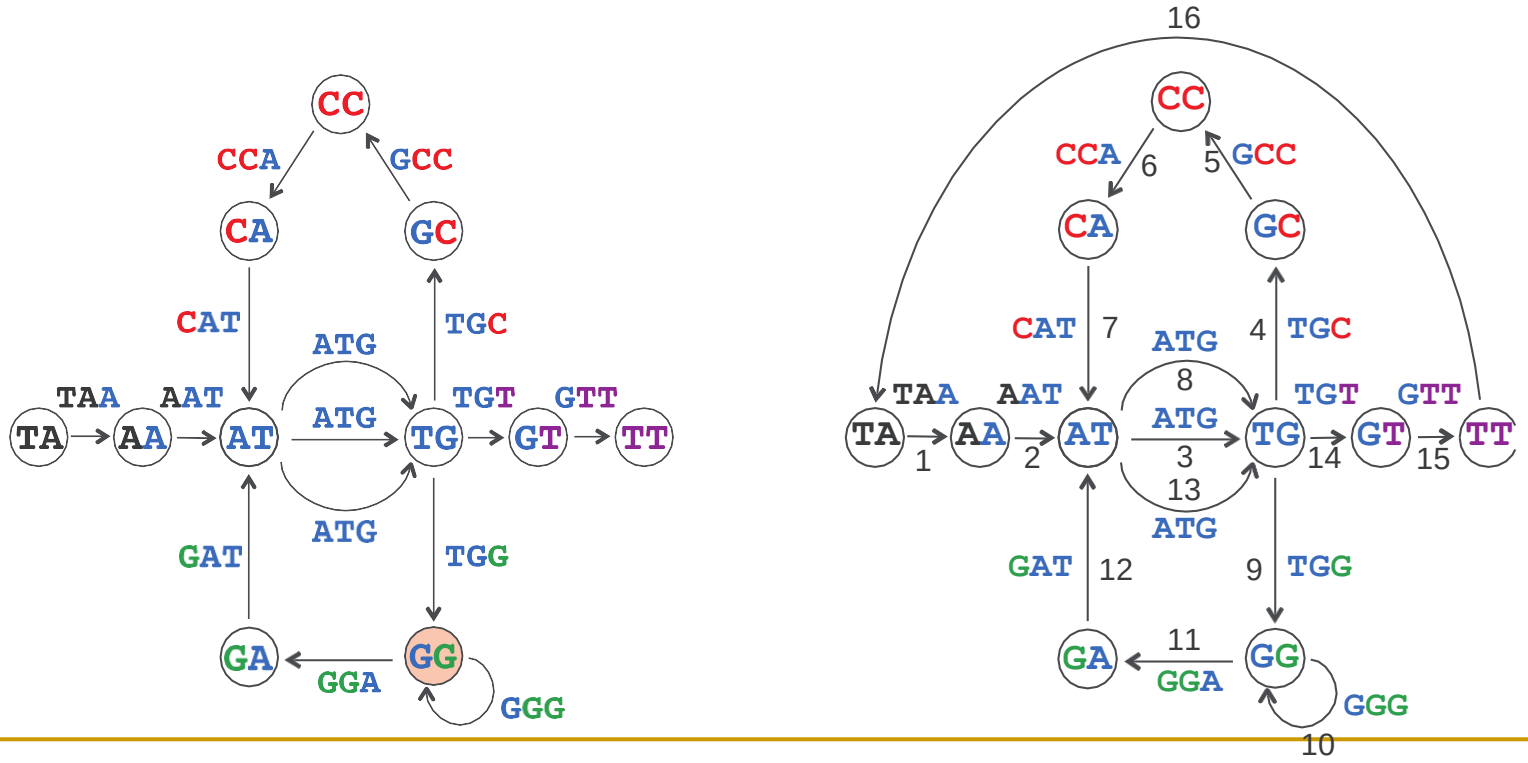
STOP: How do we find an Eulerian *path* in this graph?



Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

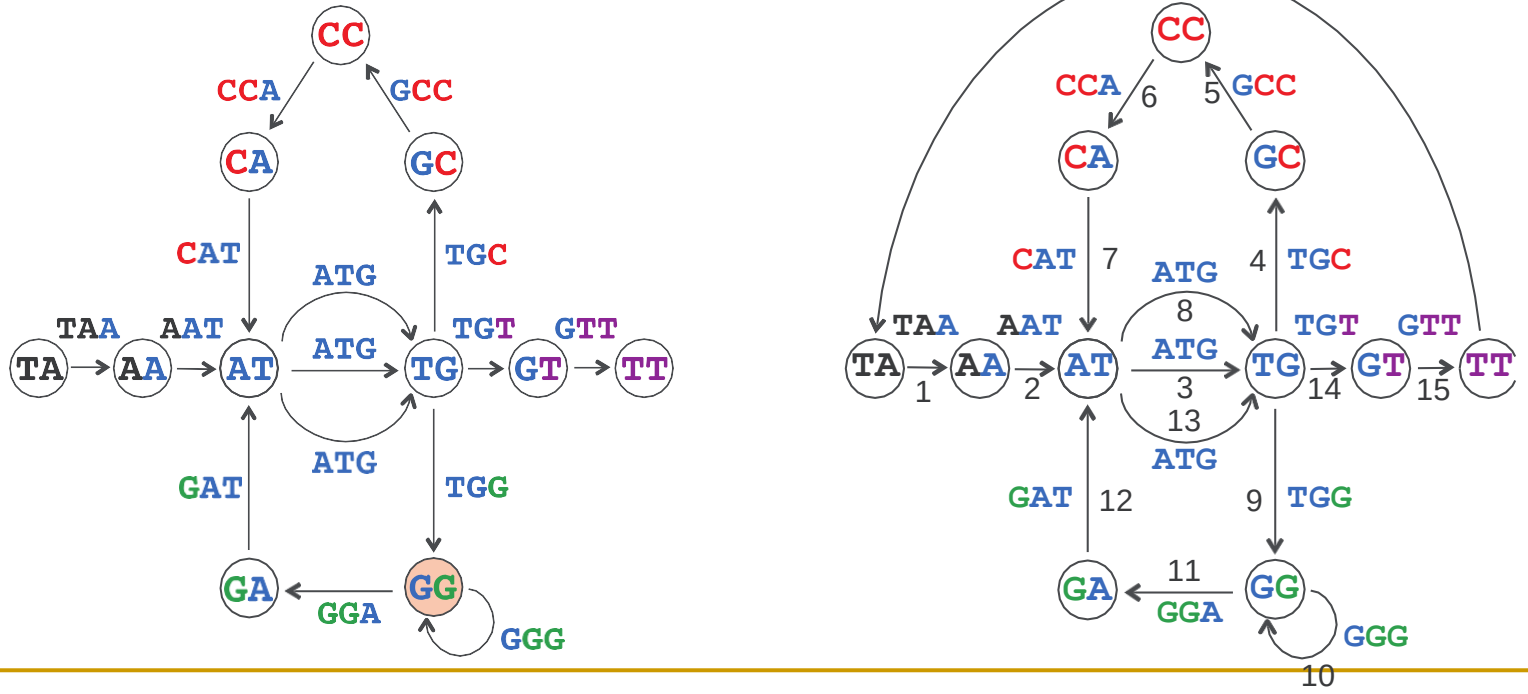
From Eulerian Cycles to Paths

Answer: Simply draw an edge connecting the two unbalanced nodes to form a balanced graph. Eulerian cycle on right = Eulerian path on left.



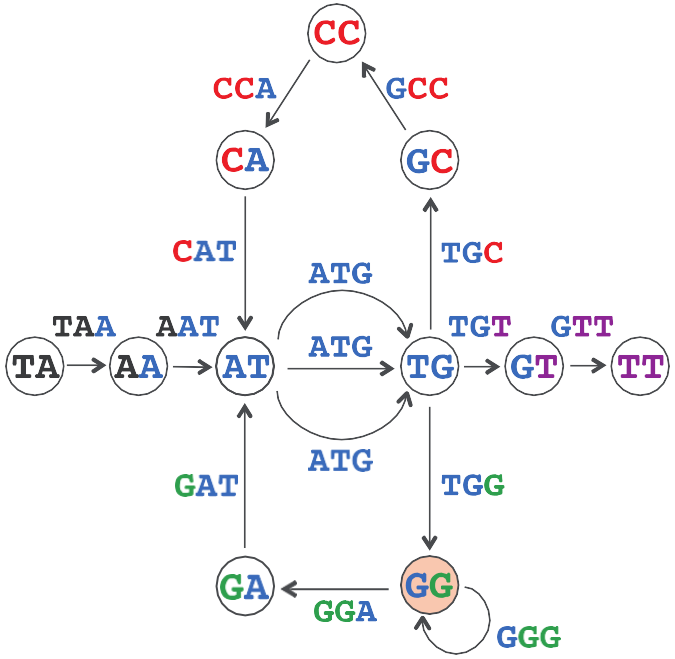
From Eulerian Cycles to Paths

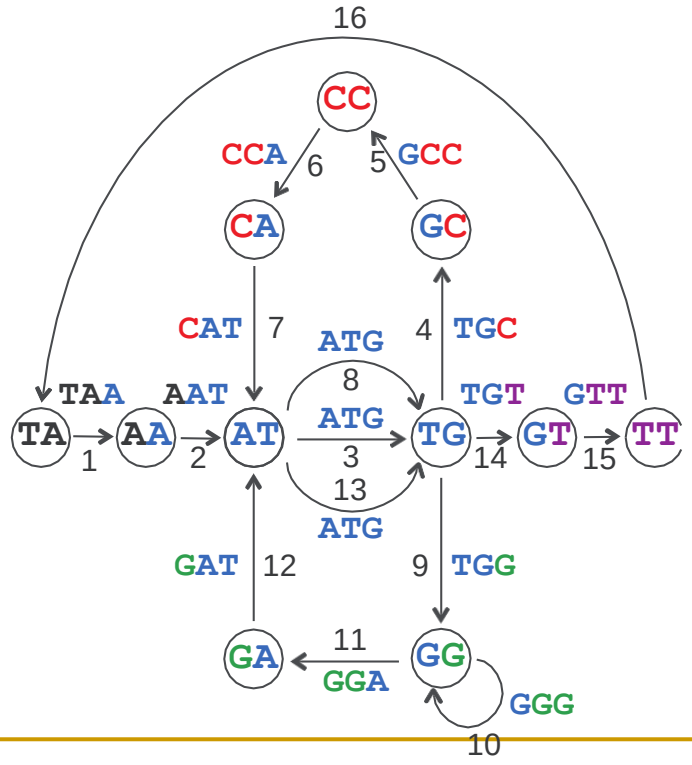
STOP: Why will the augmented de Bruijn graph on the right be balanced for any collection of strings *Patterns*?



From Eulerian Cycles to Paths

Answer: For every node *v* in de Bruijn graph, Indegree(*v*) and Outdegree(*v*) are both equal to # of patterns containing *v* as prefix/suffix, respectively.





We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*

Input: An integer *k* and a collection *Patterns* of *k*-mers.

Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*

Input: An integer *k* and a collection *Patterns* of *k*-mers.

Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*

Input: An integer *k* and a collection *Patterns* of *k*-mers.

Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*

Input: An integer *k* and a collection *Patterns* of *k*-mers.

Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.
4. Infer Eulerian path in *G* from this cycle.

We Can Assemble a Genome!

String Reconstruction Problem: *Reconstruct a string from its k-mer composition.*

Input: An integer *k* and a collection *Patterns* of *k*-mers.

Output: A string *Text* with *k*-mer composition equal to *Patterns* (if such a string exists).

1. Form de Bruijn graph *G* from *Patterns*.
2. Add edge to make modified graph *G'* balanced.
3. Find Eulerian cycle in *G'*.
4. Infer Eulerian path in *G* from this cycle.
5. Convert "genome path" into string *Text*.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

de Bruijn Assembly in Research

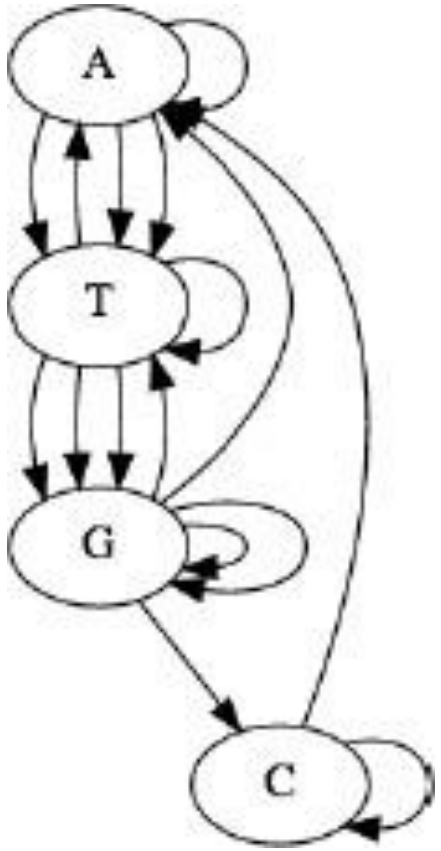
An Eulerian path approach to DNA fragment assembly | PNAS
https://www.pnas.org/content/98/17/9748
by PA Pevzner - 2001 - Cited by 1230 - Related articles
Aug 14, 2001 - Pavel A. **Pevzner**, Haixu Tang, and Michael S. Waterman ... Our new **euler** algorithm is very different from this natural ..... **Google Scholar**.

Velvet: Algorithms for de novo short read assembly using de Bruijn ...
https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2336801/
by DR Zerbino - 2008 - Cited by 7502 - Related articles
Mar 17, 2008 - **Velvet** represents a new approach to assembly that can leverage very short reads in ... The **EULER** assembler (Pevzner et al. ... s 2001 **paper**.
Abstract · Results · Discussion · Methods

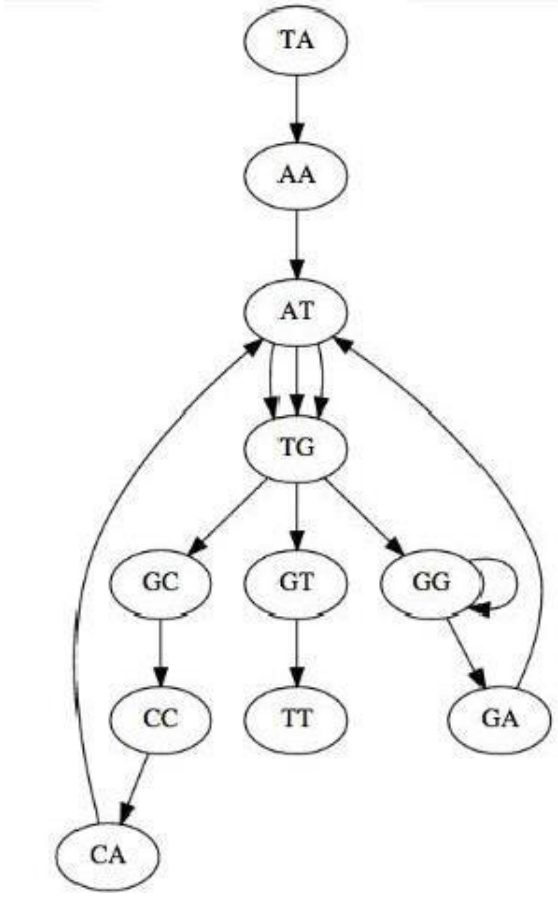
SPAdes: A New Genome Assembly Algorithm and Its ... - NCBI - NIH
https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3342519/
by A Bankevich - 2012 - Cited by 4624 - Related articles
We present the **SPAdes assembler**, introducing a number of new algorithmic ... De Bruijn graphs, PDBGs, and several other graphs in this **paper** are special ...
Assembly in SPAdes: An ... · Stage 2: Utilization of ... · Additional Details on ...

Longer Reads Means More Accuracy

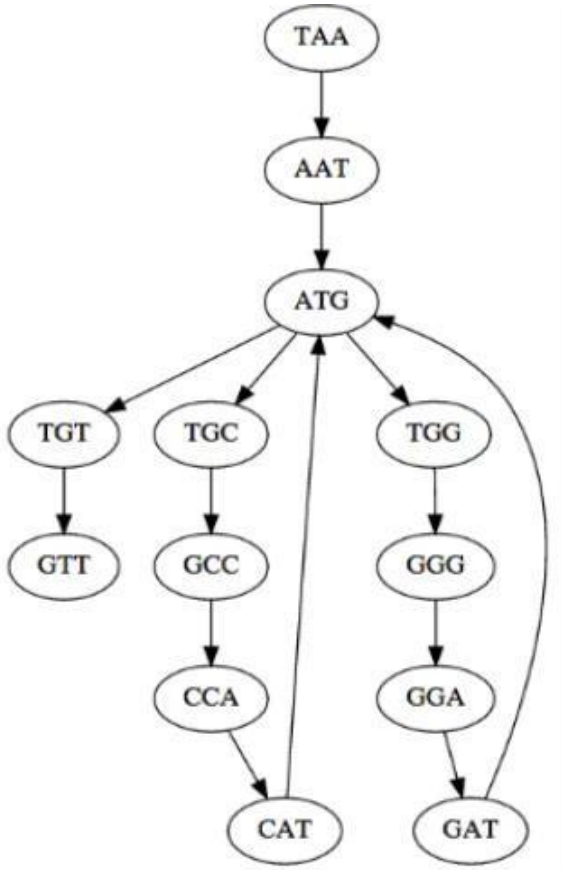
DeBruijn2(TAATGCCATGGGATGTT)



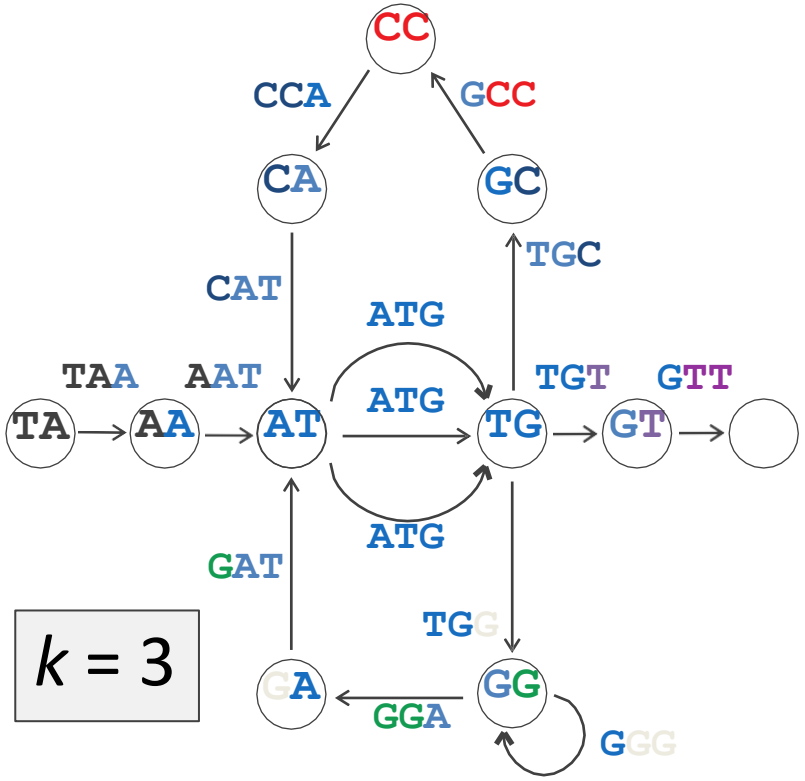
DeBruijn3(TAATGCCATGGGATGTT)

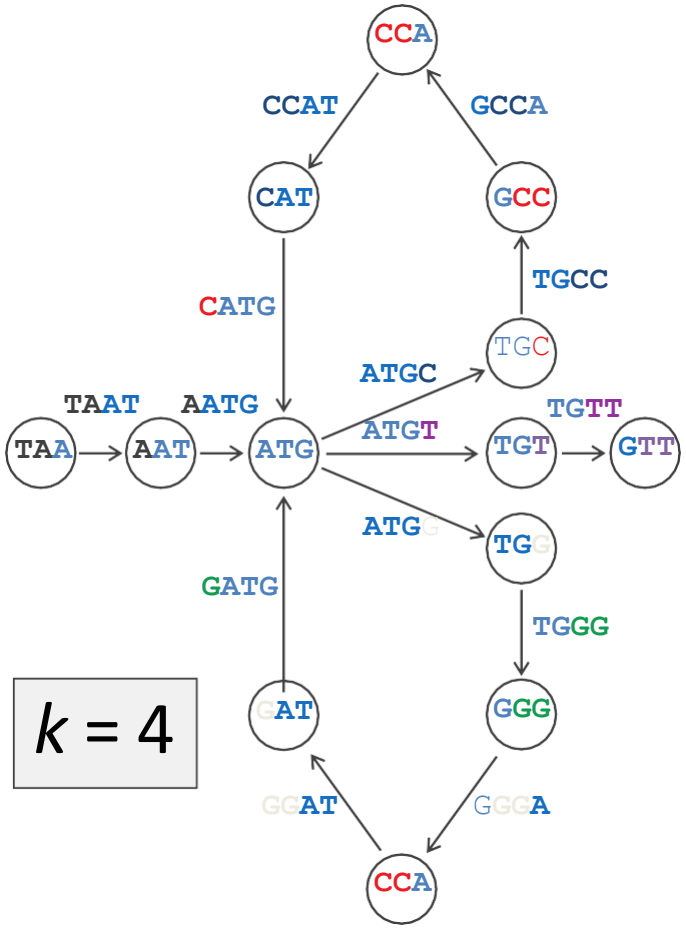


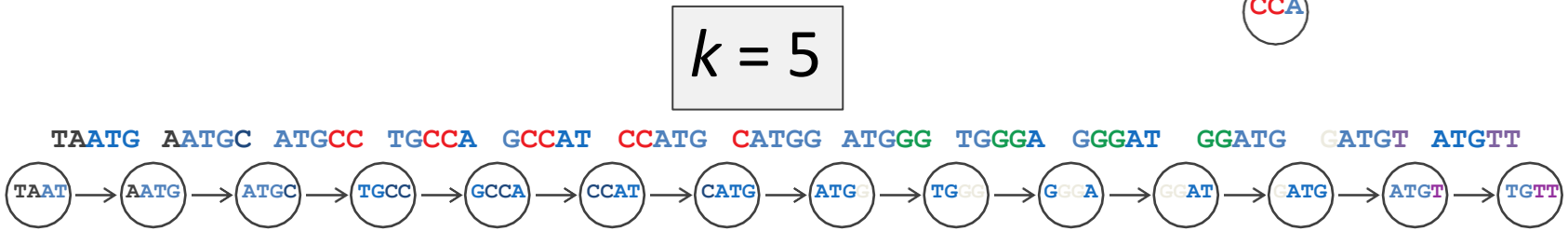
DeBruijn4(TAATGCCATGGGATGTT)



Longer Reads Means More Accuracy







Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Complications of Genome Assembly

1. Reads have **imperfect "coverage"** of the underlying genome – there may be some regions that are not covered by any reads.

2. Sequencing machines are **error-prone**.

3. Exact multiplicities of *k*-mers are **unknown**.

4. DNA is **double-stranded** (and may consist of **multiple chromosomes**).

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.

Boosting Coverage through Read Breaking

```
ATGCCGTATGGACAACGACT
ATGCCGTATG
  GCCGTATGGA
    GTATGGACAA
      GACAACGACT
```

```
ATGCCGTATGGACAACGACT
ATGCC
 TGCCG
  GCCGT
  CGTAT
   GTATG
    TATGG
    ATGGA
     TGGAC
     GGACA
     GACAA
     ACAAC
      CAACG
       AACGA
       ACGAC
        CGACT
```

Read breaking: Split each read into all its *k*-mer substrings (for a smaller value of *k*).

Boosting Coverage through Read Breaking

```
ATGCCGTATGGACAACGACT
ATGCCGTATG
  GCCGTATGGA
    GTATGGACAA
      GACAACGACT
```

```
ATGCCGTATGGACAACGACT
ATGCC
 TGCCG
  GCCGT
  CGTAT
   GTATG
    TATGG
    ATGGA
     TGGAC
     GGACA
     GACAA
     ACAAC
      CAACG
       AACGA
       ACGAC
        CGACT
```

Note: The smaller value of *k*, the higher our coverage will be, but also the more repeats and the more "tangled" our graph.

Bioinformatics Algorithms: An Active Learning Approach. © 2020 Compeau and Pevzner.